AI Assisted Coding
Lab Assignment - 9.1

Name : Shivanand Rama
Hall.t.No. : 2303A51037
Batch No. : 01

Lab Experiment: Documentation Generation -Automatic documentation and code comments

Lab Objectives

1. To understand automatic documentation generation.
2. To generate code comments and docstrings using AI tools.
3. To learn the importance of documentation in software development.

Lab Outcomes

1. Students will be able to generate documentation automatically for code.
2. Students will be able to add clear comments and docstrings to programs.
3. Students will be able to improve code readability and maintainability using documentation.

Problem 1:

Consider the following Python function:

```
def find_max(numbers):
    return max(numbers)
```

Task:

- Write documentation for the function in all three formats:
  - (a) Docstring
  - (b) Inline comments
  - (c) Google-style documentation

- Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
- Recommend which documentation style is most effective for a mathematical utilities library and justify your Answer.

Copilot Suggested Code :

task_1.py > ...

```python
# Consider the following Python function:
# def find_max(numbers):
# return max(numbers)
# Write documentation for the function in all three formats:
# (a) Docstring
# (b) Inline comments
# (c) Google-style documentation

# (a) Docstring
def find_max(numbers):
    """
    Finds the maximum value in a list of numbers.

    Args:
        numbers (list): A list of numerical values.
    Returns:
        The maximum value from the list of numbers.
    Raises:
        ValueError: If the input list is empty.
    """
    if not numbers:
        raise ValueError("The input list cannot be empty.")
    return max(numbers)
# (b) Inline comments
def find_max(numbers):
    # Check if the input list is empty and raise an error if it is
    if not numbers:
        raise ValueError("The input list cannot be empty.")

    # Use the built-in max function to find and return the maximum value
    return max(numbers)
# (c) Google-style documentation
def find_max(numbers):
    """
    Finds the maximum value in a list of numbers.

    Args:
        numbers (list): A list of numerical values.
    Returns:
        The maximum value from the list of numbers.
    Raises:
        ValueError: If the input list is empty.
    """
    if not numbers:
        raise ValueError("The input list cannot be empty.")
    return max(numbers)
```

Command used :

```
PS C:\AI Assist\lab_assignment-9.1> & "C:\Program Files\Python314\python.exe" "c:/AI Assist/lab_assignment-9.1/task_1.py"
PS C:\AI Assist\lab_assignment-9.1> python -m pydoc task_1
Help on module task_1:

NAME
    task_1

DESCRIPTION
    # Consider the following Python function:
    #   def find_max(numbers):
    #       return max(numbers)
    # Write documentation for the function in all three formats:
    # (a) Docstring
    # (b) Inline comments
    # (c) Google-style documentation

FUNCTIONS
    find_max(numbers)
        Finds the maximum value in a list of numbers.

        Args:
            numbers (list): A list of numerical values.
        Returns:
            The maximum value from the list of numbers.
        Raises:
            ValueError: If the input list is empty.

FILE
    c:\ai assist\lab_assignment-9.1\task_1.py


PS C:\AI Assist\lab_assignment-9.1> python -m pydoc -w task_1
wrote task_1.html
PS C:\AI Assist\lab_assignment-9.1> python -m pydoc -p 8080
Server ready at http://localhost:8080/
Server commands: [b]rowser, [q]uit
server> b
server>
```

After typing these commands an html file is generated :

```
task_1.html > html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Python: module task_1</title>
</head><body>

<table class="heading">
<tr class="heading-text decor">
<td class="title"> <br><strong class="title">task_1</strong></td>
<td class="extra"><a href=".">index</a><br><a href="file:c%3A%5Cai%20assist%5Clab_assignment-9.1%5Ctask_1.py">c:\ai assist\lab_assignment-9.1\task_1.py</a></td></tr></tab
    <p><span class="code"># Consider the following Python function:<br>
#    def <a href="#-find_max">find_max</a>(numbers):<br>
#        return max(numbers)<br>
# Write documentation for the function in all three formats:<br>
# (a) Docstring<br>
# (b) Inline comments<br>
# (c) Google-style documentation</span></p>
<p>
<table class="section">
<tr class="decor functions-decor heading-text">
<td class="section-title" colspan=3> <br><strong class="bigsection">Functions</strong></td></tr>

<tr><td class="decor functions-decor"><span class="code">       </span></td><td> </td>
<td class="singlecolumn"><dl><dt><a name="-find_max"><strong>find_max</strong></a>(numbers)</dt><dd><span class="code">Finds the maximum value in 
 <br>
Args:<br>
    numbers (list): A list of numerical values.<br>
Returns:<br>
    The maximum value from the list of numbers.<br>
Raises:<br>
    ValueError: If the input list is empty.</span></dd></dl>
</td></tr></table>
</body></html>
```

The output of this html file is :

index
task_1 c:\ai assist\lab_assignment-9.1\task_1.py

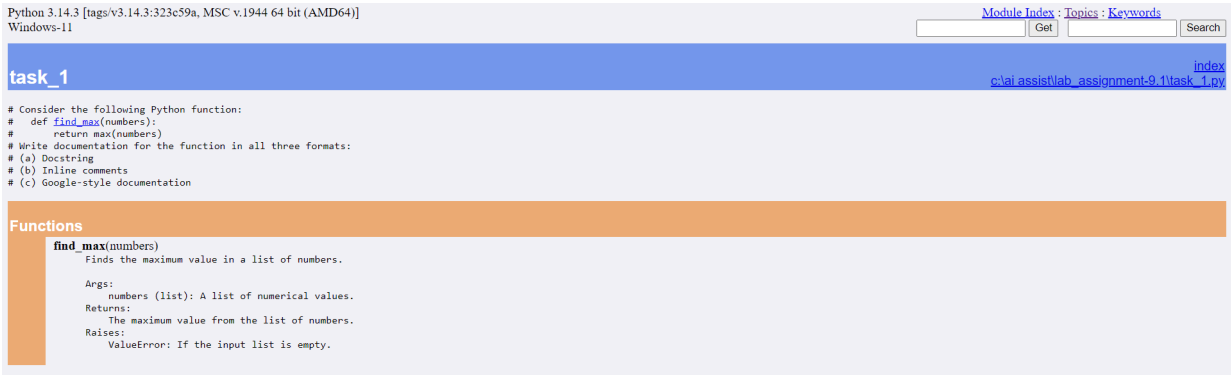
```
# Consider the following Python function:
#   def find_max(numbers):
#       return max(numbers)
# Write documentation for the function in all three formats:
# (a) Docstring
# (b) Inline comments
# (c) Google-style documentation
```

Functions

find_max(numbers)
Finds the maximum value in a list of numbers.

Args:
numbers (list): A list of numerical values.
Returns:
The maximum value from the list of numbers.
Raises:
ValueError: If the input list is empty.

With the html file a webpage is also created with the commands we typed earlier :



Problem 2:

Consider the following Python function:

```
def login(user, password, credentials):
    return credentials.get(user) == password
```

Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

Copilot Suggested Code :

```python
task_2.py > ...
# Consider the following Python function:
#   def login(user, password, credentials):
#       return credentials.get(user) == password

# Documentation in three formats:
# a) Docstring format:
def login(user, password, credentials):
    """
    Authenticates a user by comparing the provided password with the stored credentials.

    Parameters:
    user (str): The username of the user trying to log in.
    password (str): The password provided by the user.
    credentials (dict): A dictionary containing usernames as keys and their corresponding passwords as values.

    Returns:
    bool: True if the credentials match, False otherwise.
    """
    return credentials.get(user) == password
# b) Inline comments format:
def login(user, password, credentials):
    # Get the stored password for the given user from the credentials dictionary
    stored_password = credentials.get(user)

    # Compare the provided password with the stored password and return the result
    return stored_password == password
# c) External documentation format (e.g., README.md):
# Function: login
# Description: Authenticates a user by comparing the provided password with the stored credentials.
# Parameters:
# - user (str): The username of the user trying to log in.
# - password (str): The password provided by the user.
# - credentials (dict): A dictionary containing usernames as keys and their corresponding passwords as values.
# Returns:
# - bool: True if the credentials match, False otherwise.

# Critical comparison of approaches:
# - Docstring format is beneficial for providing detailed information about the function, including parameters and return
#   values. It is easily accessible within the code and can be used by tools like IDEs for auto-completion and documentation generation.
# - Inline comments are useful for explaining specific lines of code or logic, but they can become cluttered and may not provide a comprehensive overview of the function's purpose and usage.
# - External documentation is ideal for providing a high-level overview of the function and its usage, but it may not be as easily accessible as docstrings or inline comments, especially for developers who are new to the project.

# Recommendation:
# For new developers onboarding a project, the docstring format would be the most helpful. It provides a clear and concise explanation of the function's purpose, parameters, and return values directly within the code. This allows new developers
# to quickly understand how to use the function without needing to refer to external documentation or sift through inline comments. Additionally, many development tools can utilize docstrings for better code navigation
# and auto-completion, further enhancing the onboarding experience.
```

Command used :

```
PS C:\AI Assist\lab_assignment-9.1> & "C:\Program Files\Python314\python.exe" "c:/AI Assist/lab_assignment-9.1/task_2.py"
PS C:\AI Assist\lab_assignment-9.1> python -m pydoc task_2
Help on module task_2:

NAME
    task_2

DESCRIPTION
    # Consider the following Python function:
    #   def login(user, password, credentials):
    #       return credentials.get(user) == password

FUNCTIONS
    login(user, password, credentials)
        # b) Inline comments format:

FILE
    c:\ai assist\lab_assignment-9.1\task_2.py


PS C:\AI Assist\lab_assignment-9.1> python -m pydoc -w task_2
wrote task_2.html
PS C:\AI Assist\lab_assignment-9.1> python -m pydoc -p 8080
Server ready at http://localhost:8080/
Server commands: [b]rowser, [q]uit
server> b
server>
```

After typing these commands an html file is generated :

```html
task_2.html > ...
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Python: module task_2</title>
</head><body>

<table class="heading">
<tr class="heading-text decor">
<td class="title"> <br><strong class="title">task_2</strong></td>
<td class="extra"><a href=".">index</a><br><a href="file:c%3A%5Cai%20assist%5Clab_assignment-9.1%5Ctask_2.py">c:\ai assist\lab_assignment-9.1\task_2.py</a></td></tr></table>
    <p><span class="code"># Consider the following Python function:<br>
#   def <a href="#-login">login</a>(user, password, credentials):<br>
#       return credentials.get(user) == password</span></p>
<p>
<table class="section">
<tr class="decor functions-decor heading-text">
<td class="section-title" colspan=3> <br><strong class="bigsection">Functions</strong></td></tr>

<tr><td class="decor functions-decor"><span class="code">       </span></td><td> </td>
<td class="singlecolumn"><dl><dt><a name="-login"><strong>login</strong></a>(user, password, credentials)</dt><dd><span class="code"># b) Inline comments format:</span></dd></dl>
</td></tr></table>
</body></html>
```

The output of this html file is :

index
task_2 c:\ai assist\lab_assignment-9.1\task_2.py

```
# Consider the following Python function:
#   def login(user, password, credentials):
#       return credentials.get(user) == password
```

Functions

login(user, password, credentials)
b) Inline comments format:

With the html file a webpage is also created with the commands we typed earlier :

Python 3.14.3 [tags/v3.14.3:323c59a, MSC v.1944 64 bit (AMD64)]
Windows-11

Module Index : Topics : Keywords
Get Search

task_2 index c:\ai assist\lab_assignment-9.1\task_2.py

```
# Consider the following Python function:
#   def login(user, password, credentials):
#       return credentials.get(user) == password
```

Functions

login(user, password, credentials)
b) Inline comments format:

Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named calculator.py and demonstrate automatic documentation generation.

Instructions:

1. Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings:

- add(a, b) – returns the sum of two numbers
- subtract(a, b) – returns the difference of two numbers
- multiply(a, b) – returns the product of two numbers
- divide(a, b) – returns the quotient of two numbers

2. Display the module documentation in the terminal using Python's documentation tools.
3. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

Copilot Suggested Code :

task_3.py > ...

```python
# Write a Python module named calculator.py and demonstrate automatic documentation
# generation.

def add(a, b):
    """
    Returns the sum of two numbers.

    Parameters:
    a (float): The first number.
    b (float): The second number.

    Returns:
    float: The sum of a and b.
    """
    return a + b

def subtract(a, b):
    """
    Returns the difference of two numbers.

    Parameters:
    a (float): The first number.
    b (float): The second number.

    Returns:
    float: The difference of a and b.
    """
    return a - b

def multiply(a, b):
    """
    Returns the product of two numbers.

    Parameters:
    a (float): The first number.
    b (float): The second number.

    Returns:
    float: The product of a and b.
    """
    return a * b

def divide(a, b):
    """
    Returns the quotient of two numbers.

    Parameters:
    a (float): The first number.
    b (float): The second number.

    Returns:
    float: The quotient of a and b.

    Raises:
    ValueError: If b is zero.
    """
    if b == 0:
        raise ValueError("Cannot divide by zero.")
    return a / b

# To display the module documentation in the terminal, you can use the built-in
# help function:
if __name__ == "__main__":
    import pydoc
    print(pydoc.render_doc("calculator"))
```

Command used :

```
PS C:\B.Tech\3-2\AI Assit Code\lab_assignment-9.1> python -m pydoc task_3
Help on module task_3:

NAME
    task_3

DESCRIPTION
    # Write a Python module named calculator.py and demonstrate automatic documentation
    # generation.

FUNCTIONS
    add(a, b)
        Returns the sum of two numbers.

        Parameters:
        a (float): The first number.
        b (float): The second number.

        Returns:
        float: The sum of a and b.

    divide(a, b)
        Returns the quotient of two numbers.

        Parameters:
        a (float): The first number.
        b (float): The second number.

        Returns:
        float: The quotient of a and b.

        Raises:
        ValueError: If b is zero.

    multiply(a, b)
        Returns the product of two numbers.

        Parameters:
        a (float): The first number.
        b (float): The second number.

        Returns:
        float: The product of a and b.

    subtract(a, b)
        Returns the difference of two numbers.

        Parameters:
        a (float): The first number.
        b (float): The second number.

        Returns:
        float: The difference of a and b.

FILE
    c:\b.tech\3-2\ai assit code\lab_assignment-9.1\task_3.py


PS C:\B.Tech\3-2\AI Assit Code\lab_assignment-9.1> python -m pydoc -w task_3
wrote task_3.html
PS C:\B.Tech\3-2\AI Assit Code\lab_assignment-9.1> python -m pydoc -p 8080
Server ready at http://localhost:8080/
Server commands: [b]rowser, [q]uit
server> b
server>
```

After typing these commands an html file is generated :

task_3.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Python: module task_3</title>
</head><body>

<table class="heading">
<tr class="heading-text decor">
<td class="title"> <br><strong class="title">task_3</strong></td>
<td class="extra"><a href=".">index</a><br><a href="file:c%3A%5Cb.tech%5C3-2%5Cai%20assit%20code%5Clab_assignment-9.1%5Ctask_3.py">c:\b.tech\3-2\ai assit code\lab_assignment-9.1\task_3.py</a></td></tr></table>
    <p><span class="code"># Write a Python module named calculator.py and demonstrate automatic documentation <br>
# generation.</span></p>
<p>
<table class="section">
<tr class="decor functions-decor heading-text">
<td class="section-title" colspan=3> <br><strong class="bigsection">Functions</strong></td></tr>

<tr><td class="decor functions-decor"><span class="code">       </span></td><td> </td>
<td class="singlecolumn"><dl><dt><a name="-add"><strong>add</strong></a>(a, b)</dt><dd><span class="code">Returns the sum of two numbers.<br>
 <br>
Parameters:<br>
a (float): The first number.<br>
b (float): The second number.<br>
 <br>
Returns:<br>
float: The sum of a and b.</span></dd></dl>
 <dl><dt><a name="-divide"><strong>divide</strong></a>(a, b)</dt><dd><span class="code">Returns the quotient of two numbers.<br>
 <br>
Parameters:<br>
a (float): The first number.<br>
b (float): The second number.<br>
 <br>
Returns:<br>
float: The quotient of a and b.<br>
 <br>
Raises:<br>
ValueError: If b is zero.</span></dd></dl>
 <dl><dt><a name="-multiply"><strong>multiply</strong></a>(a, b)</dt><dd><span class="code">Returns the product of two numbers.<br>
 <br>
Parameters:<br>
a (float): The first number.<br>
b (float): The second number.<br>
 <br>
Returns:<br>
float: The product of a and b.</span></dd></dl>
 <dl><dt><a name="-subtract"><strong>subtract</strong></a>(a, b)</dt><dd><span class="code">Returns the difference of two numbers.<br>
 <br>
Parameters:<br>
a (float): The first number.<br>
b (float): The second number.<br>
 <br>
Returns:<br>
float: The difference of a and b.</span></dd></dl>
</td></tr></table>
</body></html>
```

The output of this html file is :

index
task_3 c:\b.tech\3-2\ai assit code\lab_assignment-9.1\task_3.py

Write a Python module named calculator.py and demonstrate automatic documentation
generation.

Functions

add(a, b)
Returns the sum of two numbers.

Parameters:
a (float): The first number.
b (float): The second number.

Returns:
float: The sum of a and b.

divide(a, b)
Returns the quotient of two numbers.

Parameters:
a (float): The first number.
b (float): The second number.

Returns:
float: The quotient of a and b.

Raises:
ValueError: If b is zero.

multiply(a, b)
Returns the product of two numbers.

Parameters:
a (float): The first number.
b (float): The second number.

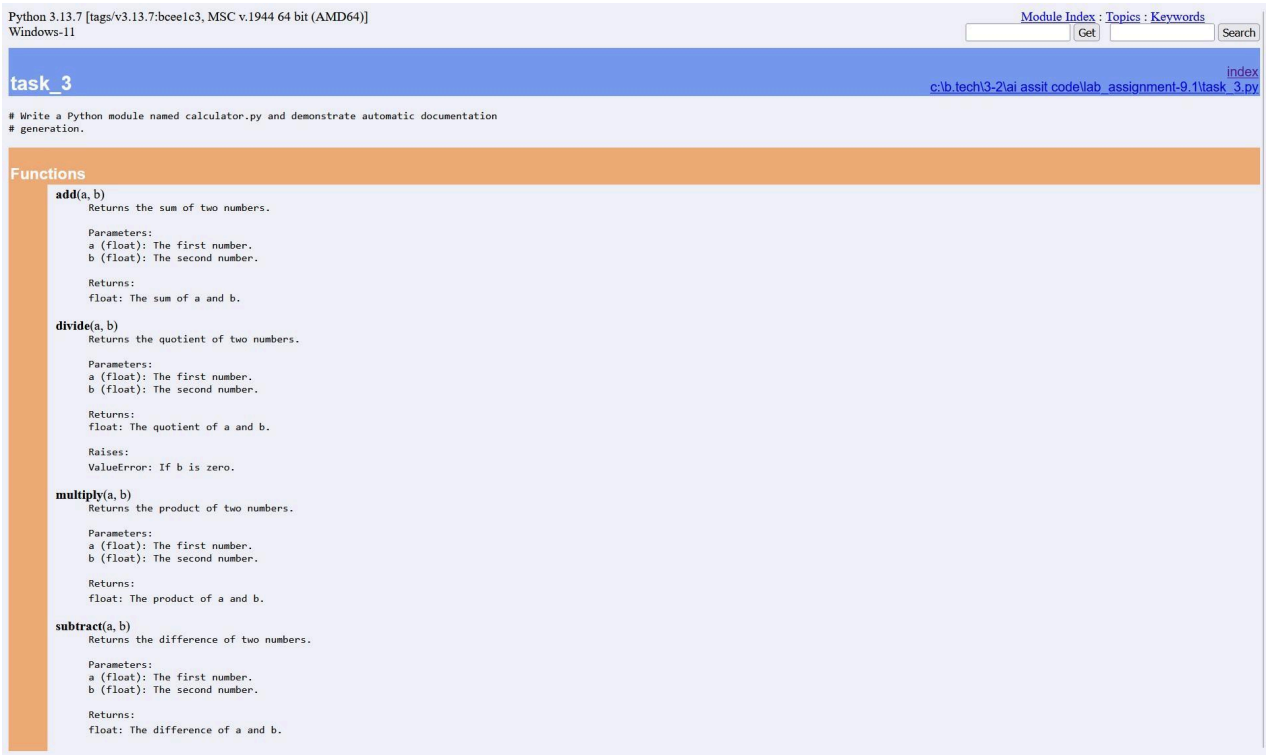
Returns:
float: The product of a and b.

subtract(a, b)
Returns the difference of two numbers.

Parameters:
a (float): The first number.
b (float): The second number.

Returns:
float: The difference of a and b.

With the html file a webpage is also created with the commands we typed earlier :



Python 3.13.7 [tags/v3.13.7:bcee1c3, MSC v.1944 64 bit (AMD64)]
Windows-11

Module Index : Topics : Keywords
Get Search

task_3

index
c:\b.tech\3-2\ai assit code\lab_assignment-9.1\task_3.py

```
# Write a Python module named calculator.py and demonstrate automatic documentation
# generation.
```

Functions

add(a, b)
Returns the sum of two numbers.

Parameters:
a (float): The first number.
b (float): The second number.

Returns:
float: The sum of a and b.

divide(a, b)
Returns the quotient of two numbers.

Parameters:
a (float): The first number.
b (float): The second number.

Returns:
float: The quotient of a and b.

Raises:
ValueError: If b is zero.

multiply(a, b)
Returns the product of two numbers.

Parameters:
a (float): The first number.
b (float): The second number.

Returns:
float: The product of a and b.

subtract(a, b)
Returns the difference of two numbers.

Parameters:
a (float): The first number.
b (float): The second number.

Returns:
float: The difference of a and b.

Problem 4:

Conversion Utilities Module

Task:

1. Write a module named conversion.py with functions:
   - o decimal_to_binary(n)
   - o binary_to_decimal(b)
   - o decimal_to_hexadecimal(n)
2. Use Copilot for auto-generating docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

Copilot Suggested Code :

task_4.py > ...

```python
# Write a module named conversion.py with functions:
# decimal_to_binary(n)
# binary_to_decimal(b)
# decimal_to_hexadecimal(n)

# conversion.py
def decimal_to_binary(n):
    """
    Convert a decimal number to its binary representation.

    Parameters:
    n (int): The decimal number to convert.

    Returns:
    str: The binary representation of the decimal number.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")
    return bin(n)[2:]
def binary_to_decimal(b):
    """Convert a binary string to its decimal representation.
    Parameters:
    b (str): The binary string to convert.
    Returns:
    int: The decimal representation of the binary string.
    """
    if not all(char in '01' for char in b):
        raise ValueError("Input must be a valid binary string.")
    return int(b, 2)
def decimal_to_hexadecimal(n):
    """
    Convert a decimal number to its hexadecimal representation.
    Parameters:
    n (int): The decimal number to convert.
    Returns:
    str: The hexadecimal representation of the decimal number.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")
    return hex(n)[2:].upper()

# Example usage
if __name__ == "__main__":
    print(decimal_to_binary(10))  # Output: '1010'
    print(binary_to_decimal('1010'))  # Output: 10
    print(decimal_to_hexadecimal(255))  # Output: 'FF'
```

Command used :

```
PS C:\B.Tech\3-2\AI Assit Code\lab_assignment-9.1> & "C:\Program Files\Python313\python.exe" "c:/B.Tech/3-2/AI Assit Code/lab_assignment-9.1/task_4.py"
1010
10
FF
PS C:\B.Tech\3-2\AI Assit Code\lab_assignment-9.1> python -m pydoc task_4
Help on module task_4:

NAME
    task_4

DESCRIPTION
    # Write a module named conversion.py with functions:
    # decimal_to_binary(n)
    # binary_to_decimal(b)
    # decimal_to_hexadecimal(n)

FUNCTIONS
    binary_to_decimal(b)
        Convert a binary string to its decimal representation.
        Parameters:
        b (str): The binary string to convert.
        Returns:
        int: The decimal representation of the binary string.

    decimal_to_binary(n)
        Convert a decimal number to its binary representation.

        Parameters:
        n (int): The decimal number to convert.

        Returns:
        str: The binary representation of the decimal number.

    decimal_to_hexadecimal(n)
        Convert a decimal number to its hexadecimal representation.
        Parameters:
        n (int): The decimal number to convert.
        Returns:
        str: The hexadecimal representation of the decimal number.

FILE
    c:\b.tech\3-2\ai assit code\lab_assignment-9.1\task_4.py


PS C:\B.Tech\3-2\AI Assit Code\lab_assignment-9.1> python -m pydoc -w task_4
wrote task_4.html
PS C:\B.Tech\3-2\AI Assit Code\lab_assignment-9.1> python -m pydoc -p 8080
Server ready at http://localhost:8080/
Server commands: [b]rowser, [q]uit
server> b
server>
```

After typing these commands an html file is generated :

task_4.html > ...

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Python: module task_4</title>
</head><body>

<table class="heading">
<tr class="heading-text decor">
<td class="title"> <br><strong class="title">task_4</strong></td>
<td class="extra"><a href=".">index</a><br><a href="file:c%3A%5Cb.tech%5C3-2%5Cai%20assit%20code%5Clab_assignment-9.1%5Ctask_4.py">c:\b.tech\3-2\ai assit code\lab_assignmen
    <p><span class="code"># Write a module named conversion.py with functions:<br>
# <a href="#-decimal_to_binary">decimal_to_binary</a>(n)<br>
# <a href="#-binary_to_decimal">binary_to_decimal</a>(b)<br>
# <a href="#-decimal_to_hexadecimal">decimal_to_hexadecimal</a>(n)</span></p>
<p>
<table class="section">
<tr class="decor functions-decor heading-text">
<td class="section-title" colspan=3> <br><strong class="bigsection">Functions</strong></td></tr>

<tr><td class="decor functions-decor"><span class="code">      </span></td><td> </td>
<td class="singlecolumn"><dl><dt><a name="-binary_to_decimal"><strong>binary_to_decimal</strong></a>(b)</dt><dd><span class="code">Convert a binary string&nb
Parameters:<br>
b (str): The binary string to convert.<br>
Returns:<br>
int: The decimal representation of the binary string.</span></dd></dl>
 <dl><dt><a name="-decimal_to_binary"><strong>decimal_to_binary</strong></a>(n)</dt><dd><span class="code">Convert a decimal number to its bin
 <br>
Parameters:<br>
n (int): The decimal number to convert.<br>
 <br>
Returns:<br>
str: The binary representation of the decimal number.</span></dd></dl>
 <dl><dt><a name="-decimal_to_hexadecimal">decimal_to_hexadecimal</a>(n)</dt><dd><span class="code">Convert a decimal number to it
Parameters:<br>
n (int): The decimal number to convert.<br>
Returns:<br>
str: The hexadecimal representation of the decimal number.</span></dd></dl>
</td></tr></table>
</body></html>
```

The output of this html file is :

index

task_4 c:\b.tech\3-2\ai assit code\lab_assignment-9.1\task_4.py

Write a module named conversion.py with functions:
decimal_to_binary(n)
binary_to_decimal(b)
decimal_to_hexadecimal(n)

Functions

binary_to_decimal(b)

Convert a binary string to its decimal representation.
Parameters:
b (str): The binary string to convert.
Returns:
int: The decimal representation of the binary string.

decimal_to_binary(n)

Convert a decimal number to its binary representation.

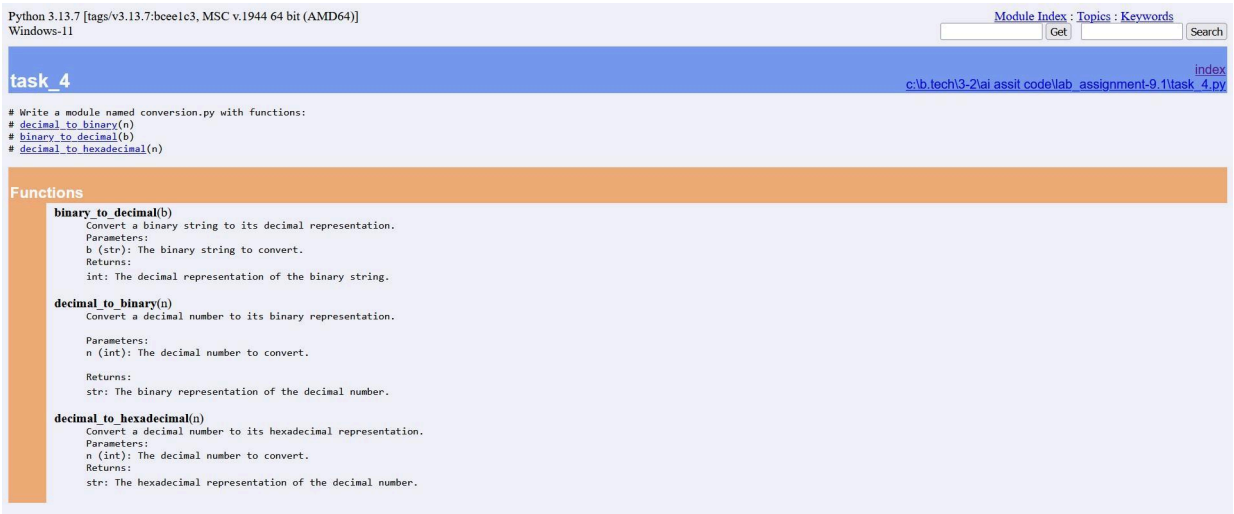
Parameters:
n (int): The decimal number to convert.

Returns:
str: The binary representation of the decimal number.

decimal_to_hexadecimal(n)

Convert a decimal number to its hexadecimal representation.
Parameters:
n (int): The decimal number to convert.
Returns:
str: The hexadecimal representation of the decimal number.

With the html file a webpage is also created with the commands we typed earlier :



Python 3.13.7 [tags/v3.13.7:bcee1c3, MSC v.1944 64 bit (AMD64)]
Windows-11

Module Index : Topics : Keywords
Get Search

task_4

index
c:\b.tech\3-2\ai assit code\lab_assignment-9.1\task_4.py

```
# Write a module named conversion.py with functions:
# decimal_to_binary(n)
# binary_to_decimal(b)
# decimal_to_hexadecimal(n)
```

Functions

binary_to_decimal(b)
Convert a binary string to its decimal representation.
Parameters:
b (str): The binary string to convert.
Returns:
int: The decimal representation of the binary string.

decimal_to_binary(n)
Convert a decimal number to its binary representation.

Parameters:
n (int): The decimal number to convert.

Returns:
str: The binary representation of the decimal number.

decimal_to_hexadecimal(n)
Convert a decimal number to its hexadecimal representation.
Parameters:
n (int): The decimal number to convert.
Returns:
str: The hexadecimal representation of the decimal number.

Problem 5 – Course Management Module

Task:

1. Create a module course.py with functions:
   - o add_course(course_id, name, credits)
   - o remove_course(course_id)
   - o get_course(course_id)
2. Add docstrings with Copilot.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a Browser.

Copilot Suggested Code :

task_5.py > ...

```python
# Create a module course.py with functions:
# add_course(course_id, name, credits)
# remove_course(course_id)
# get_course(course_id)

# course.py
class Course:
    def __init__(self):
        self.courses = {}
    def add_course(self, course_id, name, credits):
        """
        Add a course to the course management system.
        Parameters:
        course_id (str): The unique identifier for the course.
        name (str): The name of the course.
        credits (int): The number of credits for the course.
        """
        if course_id in self.courses:
            raise ValueError("Course ID already exists.")
        self.courses[course_id] = {'name': name, 'credits': credits}
    def remove_course(self, course_id):
        """
        Remove a course from the course management system.
        Parameters:
        course_id (str): The unique identifier for the course to be removed.
        """
        if course_id not in self.courses:
            raise ValueError("Course ID does not exist.")
        del self.courses[course_id]
    def get_course(self, course_id):
        """
        Get details of a specific course.
        Parameters:
        course_id (str): The unique identifier for the course to retrieve.
        Returns:
        dict: A dictionary containing the name and credits of the course.
        """
        if course_id not in self.courses:
            raise ValueError("Course ID does not exist.")
        return self.courses[course_id]
# Example usage
if __name__ == "__main__":
    course_manager = Course()
    course_manager.add_course("CS101", "Introduction to Computer Science", 4)
    print(course_manager.get_course("CS101"))  # Output: {'name': 'Introduction to Computer Science', 'credits': 4}
    course_manager.remove_course("CS101")
```

Command used :

```
PS C:\B.Tech\3-2\AI Assit Code\lab_assignment-9.1> & "C:\Program Files\Python313\python.exe" "c:/B.Tech/3-2/AI Assit Code/lab_assignment-9.1/task_5.py"
{'name': 'Introduction to Computer Science', 'credits': 4}
PS C:\B.Tech\3-2\AI Assit Code\lab_assignment-9.1> python -m pydoc task_5
Help on module task_5:

NAME
    task_5

DESCRIPTION
    # Create a module course.py with functions:
    # add_course(course_id, name, credits)
    # remove_course(course_id)
    # get_course(course_id)

CLASSES
    builtins.object
        Course

    class Course(builtins.object)
     |  # course.py
     |
     |  Methods defined here:
     |
     |  __init__(self)
     |      Initialize self.  See help(type(self)) for accurate signature.
     |
     |  add_course(self, course_id, name, credits)
     |      Add a course to the course management system.
     |      Parameters:
     |      course_id (str): The unique identifier for the course.
     |      name (str): The name of the course.
     |      credits (int): The number of credits for the course.
     |
     |  get_course(self, course_id)
     |      Get details of a specific course.
     |      Parameters:
     |      course_id (str): The unique identifier for the course to retrieve.
     |      Returns:
     |      dict: A dictionary containing the name and credits of the course.
     |
     |  remove_course(self, course_id)
     |      Remove a course from the course management system.
     |      Parameters:
     |      course_id (str): The unique identifier for the course to be removed.
     |
     |  ----------------------------------------------------------------------
     |  Data descriptors defined here:
     |
     |  __dict__
     |      dictionary for instance variables
     |
     |  __weakref__
     |      list of weak references to the object

FILE
    c:\b.tech\3-2\ai assit code\lab_assignment-9.1\task_5.py


PS C:\B.Tech\3-2\AI Assit Code\lab_assignment-9.1> python -m pydoc -w task_5
wrote task_5.html
PS C:\B.Tech\3-2\AI Assit Code\lab_assignment-9.1> python -m pydoc -p 8080
Server ready at http://localhost:8080/
Server commands: [b]rowser, [q]uit
server> b
server>
```

After typing these commands an html file is generated :

task_5.html > ...

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Python: module task_5</title>
</head><body>

<table class="heading">
<tr class="heading-text decor">
<td class="title"> <br><strong class="title">task_5</strong></td>
<td class="extra"><a href=".">index</a><br><a href="file:c%3A%5Cb.tech%5C3-2%5Cai%20assit%20code%5Clab_assignment-9.1%5Ctask_5.py">c:\b.tech\3-2\ai assit code\lab_assignment-9.1\task_5.py</a></td></tr></table>
    <p><span class="code"># Create a module course.py with functions:<br>
# add_course(course_id, name, credits)<br>
# remove_course(course_id)<br>
# get_course(course_id)</span></p>
<p>
<table class="section">
<tr class="decor index-decor heading-text">
<td class="section-title" colspan=3> <br><strong class="bigsection">Classes</strong></td></tr>

<tr><td class="decor index-decor"><span class="code">       </span></td><td> </td>
<td class="singlecolumn"><dl>
<dt class="heading-text"><a href="builtins.html#object">builtins.object</a>
</dt><dd>
<dl>
<dt class="heading-text"><a href="task_5.html#Course">Course</a>
</dt></dl>
</dd>
</dl>
 <p>
<table class="section">
<tr class="decor title-decor heading-text">
<td class="section-title" colspan=3> <br><a name="Course">class <strong>Course</strong></a>(<a href="builtins.html#object">builtins.object</a>)</td></tr>

<tr><td class="decor title-decor" rowspan=2><span class="code">    </span></td>
<td class="decor title-decor" colspan=2><span class="code"># course.py<br> </span></td></tr>
<tr><td> </td>
<td class="singlecolumn">Methods defined here:<br>
<dl><dt><a name="Course-__init__"><strong>__init__</strong></a>(self)</dt><dd><span class="code">Initialize self.  See help(type(self)) for accurate signature.</span></dd></dl>

<dl><dt><a name="Course-add_course"><strong>add_course</strong></a>(self, course_id, name, credits)</dt><dd><span class="code">Add a course to the course management system.<br>
Parameters:<br>
course_id (str): The unique identifier for the course.<br>
name (str): The name of the course.<br>
credits (int): The number of credits for the course.</span></dd></dl>

<dl><dt><a name="Course-get_course"><strong>get_course</strong></a>(self, course_id)</dt><dd><span class="code">Get details of a specific course.<br>
Parameters:<br>
course_id (str): The unique identifier for the course to retrieve.<br>
Returns:<br>
dict: A dictionary containing the name and credits of the course.</span></dd></dl>

<dl><dt><a name="Course-remove_course"><strong>remove_course</strong></a>(self, course_id)</dt><dd><span class="code">Remove a course from the course management system.<br>
Parameters:<br>
course_id (str): The unique identifier for the course to be removed.</span></dd></dl>

<hr>
Data descriptors defined here:<br>
<dl><dt><strong>__dict__</strong></dt>
<dd><span class="code">dictionary for instance variables</span></dd>
</dl>
<dl><dt><strong>__weakref__</strong></dt>
<dd><span class="code">list of weak references to the object</span></dd>
</dl>
</td></tr></table></td></tr></table>
</body></html>
```

The output of this html file is :

index
task_5 c:\b.tech\3-2\ai assit code\lab_assignment-9.1\task_5.py

```
# Create a module course.py with functions:
# add_course(course_id, name, credits)
# remove_course(course_id)
# get_course(course_id)
```

Classes

builtins.object
Course

class **Course**(builtins.object)
course.py

Methods defined here:

__init__(self)
Initialize self. See help(type(self)) for accurate signature.

add_course(self, course_id, name, credits)
Add a course to the course management system.
Parameters:
course_id (str): The unique identifier for the course.
name (str): The name of the course.
credits (int): The number of credits for the course.

get_course(self, course_id)
Get details of a specific course.
Parameters:
course_id (str): The unique identifier for the course to retrieve.
Returns:
dict: A dictionary containing the name and credits of the course.

remove_course(self, course_id)
Remove a course from the course management system.
Parameters:
course_id (str): The unique identifier for the course to be removed.

Data descriptors defined here:

__dict__
dictionary for instance variables

__weakref__
list of weak references to the object

With the html file a webpage is also created with the commands we typed earlier :

Python 3.13.7 [tags/v3.13.7:bcee1c3, MSC v.1944 64 bit (AMD64)]
Windows-11

Module Index : Topics : Keywords
Get Search

task_5

index
c:\b.tech\3-2\ai assit code\lab_assignment-9.1\task_5.py

```
# Create a module course.py with functions:
# add_course(course_id, name, credits)
# remove_course(course_id)
# get_course(course_id)
```

Classes

builtins.object
Course

class **Course**(builtins.object)

```
# course.py
```

Methods defined here:

__init__(self)

```
Initialize self.  See help(type(self)) for accurate signature.
```

add_course(self, course_id, name, credits)

```
Add a course to the course management system.
Parameters:
course_id (str): The unique identifier for the course.
name (str): The name of the course.
credits (int): The number of credits for the course.
```

get_course(self, course_id)

```
Get details of a specific course.
Parameters:
course_id (str): The unique identifier for the course to retrieve.
Returns:
dict: A dictionary containing the name and credits of the course.
```

remove_course(self, course_id)

```
Remove a course from the course management system.
Parameters:
course_id (str): The unique identifier for the course to be removed.
```

Data descriptors defined here:

__dict__

```
dictionary for instance variables
```

__weakref__

```
list of weak references to the object
```